JAVASCRIPT

ASYNCHRONE ET PROMESSES

No notes on this slide.

JAVASCRIPT

Exécution synchrone

Considérons l'exemple suivant, qui décrit à grands traits le chargement d'une carte

```
init_map();          // Initialisation de la carte
load_map_tiles();    // Chargement des tuiles de la carte, sur un serveur distant par exemple
render_map();        // Rendu de la carte
/* ... suite du code */
```

Par défaut, le JavaScript s'exécute de manière *synchrone* : ligne après ligne, chaque instruction doit être terminée avant que la suivante commence.

Dans ce code, la fonction `load_map_tiles()` va charger toutes les tuiles, faisant un grand nombre de requêtes. Cela peut prendre plusieurs secondes, pendant lesquelles la majorité du temps est passée à attendre le transfert de données.
Pendant ce temps, la page est complètement bloquée, elle a l'air de ne pas fonctionner correctement. La situation est encore pire s'il faut régulièrement charger de nouvelles tuiles.

Pourrait-on trouver une solution ?

No notes on this slide.

JAVASCRIPT

Exécution synchrone

JavaScript ne possède qu'**un seul fil d'exécution** (*thread*). Il ne fait qu'une seule chose à la fois.

```
function wait(){
    for(let i=0; i < 5_000_000_000; i++){} // Boucle de plusieurs secondes
}
wait();
console.log("Terminé !");
```

Tant que du JavaScript s'exécute, la page est "gelée" : clics, défilement, saisie au clavier, rien ne réagit. Cela est très désagréable pour la personne utilisant la page web.

Or beaucoup d'opérations prennent du temps : télécharger des tuiles, interroger un serveur WMS/WFS, attendre un clic sur la carte…
Comment attendre leur résultat sans bloquer la page ?

Un *thread* désigne un fil d'exécution. JavaScript n'utilise qu'un seul thread (excepté si on utilise des Web Workers), cela signifie que JavaScript ne peut pas exécuter plusieurs instructions en même temps, même si l'ordinateur possède un processeur comportant plusieurs cœurs.

JAVASCRIPT

Exécution asynchrone

Une opération *asynchrone* est une opération dont l'exécution ne suit pas l'ordre des lignes de code. Par exemple, la fonction `setTimeout` permet d'attendre un certain temps avant d'exécuter une fonction.

```
console.log("A");
setTimeout(() => console.log("B"), 1000); // Demande au navigateur d'exécuter
                                          // la fonction dans 1 seconde
console.log("C");
// Affiche : A, C, puis B une seconde plus tard
```

Exemples d'opérations asynchrones :

- `setTimeout`, qui indique d'attendre un certain temps
- `fetch`, qui récupère une ressource distante (images ou fichiers JSON par exemple, en indiquant une URL)
- Les événements du navigateur, "au clic", "au chargement de la page", etc.

No notes on this slide.

JAVASCRIPT

Asynchrone n'est pas parallèle

Asynchrone : plusieurs logiques à exécuter, qui passent de l'une à l'autre pendant les temps d'attente (réseau, minuteur, utilisateur).

Parallèle : plusieurs calculs exécutés réellement en même temps, sur plusieurs cœurs du processeur.

Votre code JavaScript ne s'exécute jamais en parallèle. L'asynchrone permet d'*attendre* sans bloquer, mais un calcul long bloquera toujours la page.

L'asynchrone sert la réactivité, le parallélisme sert la performance. Pour exécuter réellement un calcul lourd en parallèle dans un navigateur, il faut utiliser des *Web Workers*, qui sortent du cadre de ce cours.

JAVASCRIPT

Le problème du résultat

Comment récupérer le résultat d'une opération asynchrone ?

```
let result;
setTimeout(() => {
    result = 5;
    console.log("Log2 : ", result);
}, 1000);
console.log("Log1 : ", result);
// Reste du code
```

Ce code définit un *timeout* via `setTimeout`. La deuxième ligne s'exécute presque instantanément, et démarre l'exécution asynchrone de son contenu, dans 1 seconde.
L'affichage du Log1 s'effectue dans la foulée, `result` n'est pas encore défini, et il s'affiche `Log1 : undefined`
Une seconde plus tard, JavaScript exécute le timeout, définit la valeur de `result`, et affiche `Log2 : 5`

Conclusion : **Quand une valeur est assignée de manière asynchrone, ne pas l'utiliser en dehors.** Sinon cela provoque des comportements inattendus.

Le code qui a besoin du résultat doit donc être exécuté *après* que le résultat soit disponible. Toute la suite du chapitre présente des manières de le faire : les callbacks, les promesses, puis async / await.

JAVASCRIPT

Les callbacks

Première solution : donner à l'opération asynchrone une fonction à appeler quand le résultat est prêt. On appelle cette fonction un *callback*.

```
// Fonction fictive : télécharge les tuiles, puis appelle callback(tiles)
load_map_tiles(url, tiles => {
    console.log("Tuiles chargées : ", tiles); // Exécuté plus tard
});
console.log("La carte est interactive");      // Exécuté immédiatement
```

C'est le principe des événements, qui sont utilisés dans de nombreuses librairies, y compris en webmapping

```
map.on('click', event => console.log(event.coordinate)); // Callback appelé à chaque clic
```

Ici on définit un *callback* qui sera appelé au clic sur la carte.

No notes on this slide.

JAVASCRIPT

Les callbacks

Problème : comme le résultat d'un callback ne peut être utilisé que dans celui-ci (le résultat d'une exécution asynchrone ne peut pas être utilisé en dehors), on se retrouve avec ce type de code...

```
load_capabilities(url, (capabilities, error) => {
    if(error){ console.error(error); return; }
    load_layer(capabilities.layers[0], (layer, error) => {
        if(error){ console.error(error); return; }
        load_map_tiles(layer, (tiles, error) => {
            if(error){ console.error(error); return; }
            process_map_tiles(tiles, (result, error) => {
                if(error){ console.error(error); return; }
                display_map(result);
            });
        });
    });
});
```

C'est très difficile à lire et à modifier. On appelle cela le *callback hell*

No notes on this slide.

JAVASCRIPT

Le callback hell

Ce style de code pose plusieurs problèmes :

- Chaque étape ajoute un niveau d'indentation, le code devient difficile à lire
- La gestion des erreurs doit être répétée à chaque étape
- Ajouter, retirer ou réordonner une étape demande de modifier toute la structure
- Chaque librairie choisit sa propre convention (ordre des paramètres, gestion des erreurs, etc.)

On aimerait pouvoir écrire ces étapes *les unes à la suite des autres*

No notes on this slide.

JAVASCRIPT

Les promesses

Une *promesse* (`Promise`) est un objet qui représente le résultat futur d'une opération asynchrone.

Au lieu d'utiliser un callback, une fonction retourne immédiatement une promesse, qui est un objet "contenant" le futur résultat de l'opération.

Une promesse est toujours dans l'un de ces trois états :

- *pending* (en attente) : l'opération n'est pas terminée
- *fulfilled* (tenue) : l'opération a réussi, la promesse contient une valeur
- *rejected* (rompue) : l'opération a échoué, la promesse contient une erreur

Une promesse ne change d'état qu'une seule fois. Une fois tenue ou rompue, elle est dite *résolue* (*settled*).

No notes on this slide.

JAVASCRIPT

Les promesses

Une promesse possède une méthode `then` qui enregistre une fonction à exécuter lorsque la promesse sera tenue :

```
const p1 = load_capabilities(url);
const p2 = p1.then(capabilities => load_layer(capabilities.layers[0]))
const p3 = p2.then(layer => load_map_tiles(layer))
const p4 = p3.then(tiles => process_map_tiles(tiles))
const p5 = p4.then(result => display_map(result))
```


No notes on this slide.

JAVASCRIPT

Les promesses

On peut chaîner ces appels, pour obtenir un code beaucoup plus clair

Une promesse possède également une méthode `catch` qui enregistre une fonction à exécuter en cas d'erreur.

```
load_capabilities(url)
    .then(capabilities => load_layer(capabilities.layers[0]))
    .then(layer => load_map_tiles(layer))
    .then(tiles => process_map_tiles(tiles))
    .then(result => display_map(result))
    .catch(error => console.error(error)); // Une seule gestion des erreurs
```

Le même code que le callback hell, mais à plat : les étapes se lisent dans l'ordre.

`then` retourne lui-même une nouvelle promesse, ce qui permet d'enchaîner les appels. Si la fonction passée à `then` retourne une promesse, l'étape suivante attend sa résolution.

Une erreur levée (`throw`) dans n'importe quelle étape, ou une promesse rompue, saute toutes les étapes suivantes jusqu'au prochain `catch`. C'est l'équivalent asynchrone du bloc `try ... catch`.

JAVASCRIPT

Les promesses

Les fonctions de l'API de base en JavaScript qui prennent du temps retournent des promesses. C'est le cas de `fetch`, qui télécharge une ressource

```
const url = "https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swisssurface3c
fetch(url)                                  // Promesse de la réponse du serveur
    .then(response => {
        if(!response.ok){
            throw new Error("Erreur HTTP " + response.status);
        }
        return response.json();             // Seconde promesse, qui lit le JSON
    })
    .then(data => console.log(data))        // Quand le JSON est lu, on l'affiche
    .catch(error => console.error(error));  // Erreur réseau, HTTP ou JSON

// Le code se trouvant ici ne sera pas bloqué par les requêtes au-dessus
```

`fetch` ne rejette la promesse qu'en cas d'erreur réseau. Une réponse 404 ou 500 est considérée comme une réponse valide, d'où la vérification de `response.ok`.

JAVASCRIPT

Créer une promesse

Le constructeur `Promise` prend une fonction qui reçoit deux paramètres :

- `resolve`, la fonction à appeler quand la promesse est tenue, avec le résultat
- `reject`, facultatif, la fonction à appeler si la promesse échoue, avec l'erreur

```
// Ici on enrobe setTimeout dans une promesse
function wait(ms){
    return new Promise(resolve => setTimeout(resolve, ms));
}
wait(1000).then(() => console.log("Une seconde plus tard"));

// Transformer une fonction à callback en fonction retournant une promesse
function load_map_tiles_promise(url){
    return new Promise((resolve, reject) => {
        load_map_tiles(url, (tiles, error) => error ? reject(error) : resolve(tiles));
    });
}
```

Si une fonction retourne déjà une promesse (comme `fetch`), il ne faut pas l'envelopper dans un `new Promise`. Il suffit de retourner la promesse existante, ou la chaîne de `then`.

JAVASCRIPT

Créer une promesse

Attention, les promesses ne sont pas magiques

```
console.log("1. Avant la promesse");
new Promise(resolve => {              // Cette fonction s'exécute immédiatement,
    console.log("2. Début du calcul"); // de manière synchrone !
    for(let i=0; i < 3_000_000_000; i++){} // Boucle de plusieurs secondes
    console.log("3. Fin du calcul");
    resolve();
}).then(() => console.log("5. Callback du then"));
console.log("4. Après la promesse");
```

La fonction passée au constructeur `Promise` est exécutée **immédiatement**, pendant le `new Promise(...)`. Ici, "4. Après la promesse" ne s'affiche qu'après la fin de la boucle, et la page reste gelée pendant ce temps, exactement comme sans promesse. Seul le callback du `then` est exécuté plus tard.

Une promesse ne rend pas un code parallèle. Elle permet seulement de manipuler un résultat qui arrivera plus tard.

No notes on this slide.

JAVASCRIPT

L'ordre d'exécution

Comment JavaScript décide-t-il quand exécuter un élément asynchrone ? Grâce à la *boucle d'événements* (*event loop*)

1. Le code en cours (script principal, ou callback) s'exécute **jusqu'au bout**, sans jamais être interrompu
2. Quand JavaScript exécute une ligne avec comportement asynchrone :
    - Le navigateur prépare l'opération (attendre un certain temps, envoyer une requête, etc.)
    - Quand l'opération est prête (le temps est écoulé, la requête a obtenu une réponse, etc.), il place le code à exécuter dans la **file d'attente**
3. Quand le code en cours est terminé, la boucle d'événements prend le **prochain** code de la file d'attente, l'exécute jusqu'au bout, puis recommence

No notes on this slide.

JAVASCRIPT

Quand l'exécution s'arrête-t-elle ?

```
document.addEventListener("click", () => console.log("Clic !"));
console.log("Début");
for(let i=0; i < 3_000_000_000; i++){} // Cliquez pendant la boucle...
console.log("Fin");
// Affiche : Début, Fin, puis Clic !
```

JavaScript n'interrompt **jamais** un code en cours. Le clic a lieu pendant la boucle, mais son callback attend dans la file que le script soit terminé.

"Jusqu'au bout" signifie jusqu'à ce que la pile d'appels soit vide : la dernière ligne du script ou du callback, y compris toutes les fonctions qu'il appelle. Un autre code ne peut commencer qu'à la fin du code en cours.

Il n'y a donc jamais deux morceaux de code JavaScript qui s'exécutent "en même temps", et aucun ne peut être coupé au milieu. C'est ce qui rend l'asynchrone en JavaScript relativement simple : entre deux lignes d'un même callback, aucune variable ne peut être modifiée par un autre code.
La contrepartie : un callback trop long bloque tous les autres, et la page entière.

JAVASCRIPT

L'ordre d'exécution

Qu'affiche ce code ?

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
new Promise(resolve => {
    console.log("4");
    resolve();
}).then(() => console.log("5"));
console.log("6");
```

Ce code affichera : `1 4 6 3 5 2`

- `1`, `4`, `6` : code synchrone, exécuté jusqu'au bout (la fonction de `new Promise` en fait partie)
- `3`, `5` : callbacks de promesses, exécutés dès que le code principal est terminé
- `2` : callback du minuteur, exécuté en dernier même avec un délai de 0, car les callbacks des promesses sont prioritaires

`Promise.resolve()` crée une promesse déjà tenue. Son `then` n'est pourtant pas exécuté immédiatement : un callback de promesse est *toujours* placé dans la file d'attente, jamais exécuté de manière synchrone.
`setTimeout(f, 0)` ne signifie pas "maintenant", mais "dès que possible, dans au moins 0 ms".

JAVASCRIPT

L'ordre d'exécution

Conséquence : ne jamais attendre un résultat avec une boucle

```
let result;
fetch('https://data.geo.admin.ch/api/stac/v0.9/').then(response => { result = response; });
while(result == undefined){} // Boucle infinie !
console.log(result);
```

Le callback du `then` ne peut s'exécuter que lorsque la pile d'appels est vide. Or la boucle `while` ne se termine jamais, donc la pile n'est jamais vide, donc `result` n'est jamais assigné.
La page est gelée pour toujours.

Le code qui dépend d'un résultat asynchrone doit se trouver *dans* le `then` (ou après un `await`), jamais après une attente active.

No notes on this slide.

JAVASCRIPT

async / await

Les chaînes de `then` restent moins lisibles que du code synchrone. Il existe une syntaxe plus simple

```
async function show_map(url){
    try {
        const capabilities = await load_capabilities(url);
        const layer = await load_layer(capabilities.layers[0]);
        const tiles = await load_map_tiles(layer);
        const result = await process_map_tiles(tiles);
        display_map(result);
    } catch(error) {
        console.error(error);
    }
}
```

Le mot-clé `async` se place devant une fonction et lui fait retourner une promesse.

Le mot-clé `await` indique que le reste de la fonction s'exécutera après la résolution de la promesse qu'il précède

Le code se lit comme du code synchrone, et les erreurs se gèrent avec un `try ... catch` classique : une promesse rompue devient une exception levée par le `await`.

JAVASCRIPT

async / await

Comparaison sur un exemple réel : récupérer le titre d'un jeu de données

```
function get_data_title(url){
    return fetch(url)
        .then(response => response.json())
        .then(data => data['title']);
}
```

```
async function get_data_title(url){
    const response = await fetch(url);
    const data = await response.json();
    return data['title'];
}
```

Ces deux fonctions sont équivalentes, et s'utilisent de la même manière

```
const url = "https://data.geo.admin.ch/api/stac/v0.9/collections/ch.swisstopo.swisssurface3c
get_data_title(url).then(title => console.log("Titre : " + title))
// Affichera "Titre : swissSURFACE3D Raster, the digital surface model of Switzerland"
```


No notes on this slide.

JAVASCRIPT

async / await, un sucre syntaxique

`async` / `await` n'apporte rien de nouveau : c'est une autre écriture des promesses (un *sucre syntaxique*)

```
async function f(){
    console.log("A");
    const x = await g();
    console.log(x);
    return x * 2;
}
```

```
function f(){
    console.log("A");
    return g().then(x => {
        console.log(x);
        return x * 2;
    });
}
```

- Le code avant le premier `await` s'exécute immédiatement, de manière synchrone
- `await` place le reste de la fonction dans le `.then` de la promesse qu'il précède
- Une fonction `async` retourne toujours une promesse : `return` la tient, `throw` la rompt

La fonction est "mise en pause" au niveau du `await`, mais pas la page : la fonction retourne immédiatement sa promesse à l'appelant, qui continue son exécution. Par exemple :

```
async function f(){ console.log("A"); await null; console.log("B"); }
f();
console.log("C");
// Affiche A, C, B
```

JAVASCRIPT

Pourquoi `await` **exige** async

Imaginons qu'`await` soit autorisé dans une fonction normale

```
function get_data_title(url){              // Pas async : erreur de syntaxe
    const response = await fetch(url);
    const data = await response.json();
    return data['title'];
}
const title = get_data_title(url);         // Que vaut title ?
```

Une fonction normale doit retourner sa valeur **immédiatement**, à la fin de son exécution synchrone. Mais le titre n'arrivera que plus tard. Il n'y a que deux possibilités :

- Bloquer jusqu'à l'arrivée du résultat : c'est la boucle infinie vue précédemment, la page serait gelée
- Retourner immédiatement une promesse du résultat : c'est exactement ce que fait une fonction `async`

Le mot-clé `async` n'est donc pas un détail : il indique, à JavaScript comme à la personne qui lit le code, que la fonction retourne une promesse et non directement son résultat.
Exception : dans un module, `await` est autorisé au niveau supérieur du fichier (*top-level await*). Le module lui-même est alors traité comme asynchrone.

JAVASCRIPT

Pourquoi `await` **exige** async

Conséquence : l'asynchrone "remonte" jusqu'aux appelants

```
const title = get_data_title(url);        // Erreur fréquente : oubli du await
console.log(title);                       // Affiche "Promise { <pending> }"

async function display_title(){
    const title = await get_data_title(url); // Correct
    console.log(title);
}
```

Une fonction qui utilise `await` devient asynchrone, et ses appelants reçoivent une promesse. Ils doivent à leur tour utiliser `await` (et donc être `async`) ou `then`.

Un résultat asynchrone ne peut jamais redevenir synchrone. On ne peut que placer le code qui en dépend *après* son arrivée.

No notes on this slide.

JAVASCRIPT

async / await

Attention à ne pas attendre inutilement : des `await` successifs s'exécutent les uns après les autres

```
// Lent : la seconde requête ne démarre qu'à la fin de la première
const buildings = await (await fetch(url_buildings)).json();
const roads = await (await fetch(url_roads)).json();
```

```
// Rapide : les deux requêtes sont lancées en même temps
const [buildings, roads] = await Promise.all([
    fetch(url_buildings).then(response => response.json()),
    fetch(url_roads).then(response => response.json())
]);
```

`Promise.all` prend un tableau de promesses et retourne une promesse tenue quand **toutes** sont tenues (ou rompue dès que l'une d'elles est rompue)

Ce n'est pas du parallélisme de calcul : ce sont les *attentes* réseau qui se chevauchent. C'est particulièrement utile pour charger plusieurs couches indépendantes d'une carte.

JAVASCRIPT

Asynchrone - Références

Site officiel :

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Promise

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Statements/async_function

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Execution_model

W3Schools:

https://www.w3schools.com/js/js_async.asp

https://www.w3schools.com/js/js_async_callbacks.asp

https://www.w3schools.com/js/js_async_promises.asp

No notes on this slide.